UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Facultad de Ingeniería de Sistemas e Informática

SEMANA 5

MÉTODOS SUPERVISADOS



DOCENTE: PROF. HUGO VEGA

Inteligencia Artificial (Código: 201230703)

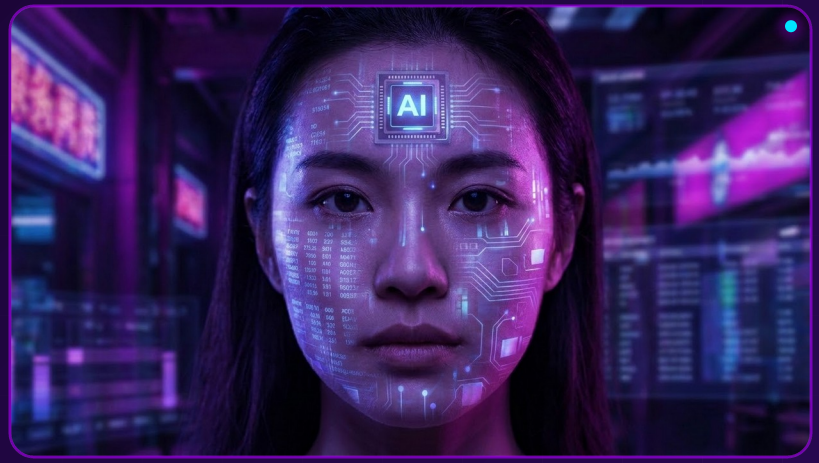
ESTUDIANTES : Calero Falcon, Gonzalo Manuel y Huanacuni Gomez Jean Carlos Josue

Semestre: 2026-II | Lima, Perú

Fundamentos de Machine Learning

El Machine Learning es una rama fundamental de la inteligencia artificial que confiere a las computadoras la capacidad de aprender patrones matemáticos a partir de datos empíricos y mejorar su desempeño de forma autónoma sin ser programadas mediante reglas fijas manuales para cada caso (Samuel, 1959).

En el software tradicional, los ingenieros deben codificar a mano todas las reglas condicionales de negocio, lo que resulta inviable cuando los fenómenos reales son complejos o cambian constantemente; en cambio, el aprendizaje inductivo recibe datos y respuestas históricas para descubrir las funciones estadísticas subyacentes que gobiernan el problema (Goodfellow et al., 2016).



> *"Un sistema aprende si su desempeño en una Tarea (T), evaluado mediante una métrica de Rendimiento (P), mejora sistemáticamente gracias a la Experiencia acumulada (E)."*
>
> — Tom Mitchell (1997)

Fundamentos del Aprendizaje Supervisado



Concepto

El aprendizaje supervisado consiste en entrenar un modelo mediante un conjunto de datos histórico donde cada muestra de entrada está explícitamente asociada a su respuesta o etiqueta correcta conocida de antemano.

(Russell & Norvig, 2021)

Clasificación

Cuando la variable a predecir es categórica y discreta.
Ejemplo: detectar transacciones fraudulentas o legítimas.

Regresión

Cuando la variable a predecir es un valor numérico continuo.
Ejemplo: estimar la demanda de energía eléctrica o el valor de un activo financiero.

¿Qué es el Aprendizaje Supervisado?



DEFINICIÓN CANÓNICA

Rama del Machine Learning cuyo objetivo es predecir una variable de salida a partir de entradas conocidas utilizando datos previamente etiquetados para entrenar el modelo (Samuel, 1959; Mitchell, 1997).

VARIABLES DEL SISTEMA

- Variables Independientes (Entradas / Features X): Representadas como una matriz de atributos.
- Variable Dependiente (Salida / Target y): Categoría discreta (clasificación) o valor continuo (regresión).

EL ROL DEL SUPERVISOR

La etiqueta histórica (label y) actúa como supervisor matemático guiando el ajuste de pesos y minimizando el error del modelo (Russell & Norvig, 2021).

DIAGRAMA DE FLUJO: APRENDIZAJE SUPERVISADO

1. FASE DE ENTRENAMIENTO (AJUSTE DE PESOS)

Entradas (X)
Matriz Features

Etiqueta (y)
Supervisor / Target

Modelo ML
`Minimización Error`

Modelo Entrenado

2. FASE DE INFERENCIA / PREDICCIÓN

Nuevas Entradas
(Datos X sin y)

Modelo Entrenado
`f(X) → y_pred`

Predicción (ŷ)
Clase / Valor

El Proceso Canónico del Aprendizaje Supervisado (Géron, 2019)

PASO 01

1. Recopilar Datos

Adquisición de dataset representativo con etiquetas confirmadas por expertos.

PASO 02

2. Entrenar Modelo

Ajuste iterativo de parámetros internos para aproximar la salida real.

PASO 03

3. Evaluar y Validar

Medición del rendimiento con métricas rigurosas sobre datos de validación independientes.

PASO 04

4. Predecir / Inferencia

Despliegue en producción ante nuevas instancias no vistas.

DIAGRAMA DE FLUJO SECUENCIAL DE LAS 4 FASES



Condiciones Técnicas de Uso Indispensables

DIAGRAMA DE REQUISITOS TÉCNICOS

Tres pilares fundamentales sin los cuales el modelo no puede aprender ni generalizar

01 CONDICIÓN 1

Datos Etiquetados Disponibles

Registro: $X_i = [x_1, x_2, \ldots, x_n]$
Etiqueta: Y_i = **label** (Target)

Cada registro de entrada debe contar obligatoriamente con su respuesta correcta (label) asociada.

Sin etiquetas explícitas es imposible efectuar el aprendizaje supervisado.

02 CONDICIÓN 2

Problema de Predicción Definido

Modelo: $f(X) \rightarrow Y$

- Discreta (Clasificación)
- Continua (Regresión)

Debe existir una formulación matemática precisa del objetivo que se pretende optimizar.

La variable objetivo debe delimitarse claramente como un valor discreto (categoría) o continuo (numérico).

03 CONDICIÓN 3

Volumen Suficiente y Representativo



Se requiere una muestra estadística significativa que cubra la variabilidad real del fenómeno.

Previene el sesgo (bias) y garantiza la capacidad de generalización en producción.

Casos de Uso (Sí) vs. Cuándo NO Usar ML Supervisado

CASOS SÍ: APLICACIÓN DE ML SUPERVISADO

1. Clasificación Médica

Diagnóstico de patologías a partir de síntomas etiquetados previamente por especialistas.

`Entrada: Síntomas (X) → Salida: Diagnóstico (Y)`

2. Predicción Continua

Estimación cuantitativa de precios de inmuebles basada en características estructuradas.

`Entrada: Características (X) → Salida: Valor $ (y)`

CASOS NO Y ALTERNATIVAS TÉCNICAS

1. Agrupamiento de Usuarios (Sin Etiquetas)

Ausencia de categorías predefinidas en los datos.

`→ Usar Clustering No Supervisado (K-Means, DBSCAN)`

2. Detección de Temas en Textos

Descubrimiento de patrones y temas latentes.

`→ Usar Topic Modeling (LDA, NMF)`

3. Generación Creativa de Contenido

Síntesis de nuevas muestras sintéticas complejas.

`→ Usar IA Generativa (GAI: Transformers, Difusión)`

El Gran Dilema: Caja Negra (Black Box) vs. Caja Blanca (White Box)

CAJA BLANCA



Transparentes e Interpretables: Modelos donde cualquier humano puede seguir la lógica interna paso a paso.

Algoritmos Característicos:

`Regresión Lineal` `Regresión Logística` `Árboles de Decisión` `k-NN (Vecinos Cercanos)`

VS

CAJA NEGRA



Opacidad Total: Arquitecturas complejas de millones de parámetros con máximo rendimiento predictivo en datos no lineales (Hastie et al., 2009).

Algoritmos Característicos:

`Deep Learning` `Random Forest` `XGBoost`

CAPÍTULO 4

Comparativa Estructural y Criterios de Selección

CAJA BLANCA

Complejidad: Baja

Rendimiento: Moderado

Interpretabilidad: Total / Alta

Depuración de Errores: Sencilla

VS

CAJA NEGRA

Complejidad: Alta

Rendimiento: Máximo

Interpretabilidad: Nula / Baja

Depuración de Errores: Muy difícil

CRITERIO DE DECISIÓN

Regla de Oro para el Ingeniero de Sistemas

Elegir Caja Negra:
Cuando el rendimiento predictivo es la prioridad absoluta sin restricciones legales de explicabilidad.

Elegir Caja Blanca:
En contextos críticos y regulados (medicina, finanzas, justicia) requeridos por auditorías y la Ley N.° 31814.

Algoritmos Básicos de Aprendizaje Supervisado

REGRESIÓN LINEAL / LOGÍSTICA

Modelado continuo y clasificación probabilística sigmoide (Hosmer et al., 2013).

ÁRBOLES CART

Reglas jerárquicas maximizando pureza Gini/Entropía (Breiman et al., 1984).

K-NEAREST NEIGHBORS (KNN)

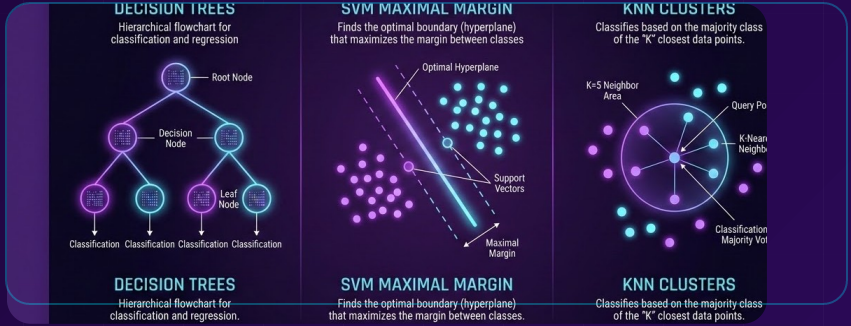
Clasificación no paramétrica por votación de vecinos (Cover & Hart, 1967).

SUPPORT VECTOR MACHINES (SVM)

Hiperplano de margen máximo con truco del kernel (Cortes & Vapnik, 1995).

NAIVE BAYES

Teorema de Bayes con independencia condicional para texto (Domingos & Pazzani, 1997).



Algoritmo 1: Regresión Lineal

PRINCIPIO MATEMÁTICO

Modela la relación entre entradas y una salida continua como una combinación lineal ponderada:

$$y = w0 + w1 \times 1 + \ldots + wdxd$$

Ajusta los parámetros minimizando la suma de errores cuadráticos mediante Mínimos Cuadrados Ordinarios (OLS) (Montgomery et al., 2021).

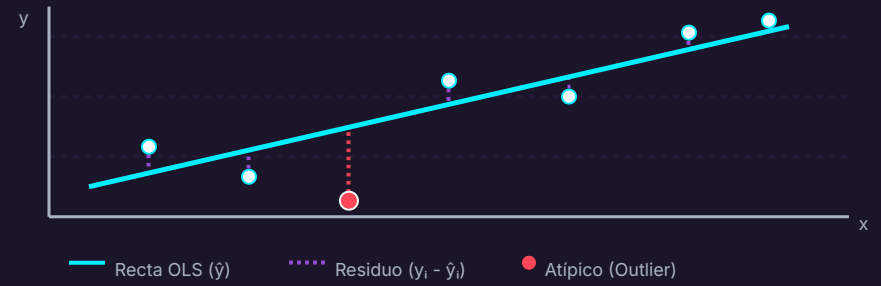
Alta Interpretabilidad: Explicación directa de la influencia de cada variable.

Sensible a Atípicos: Los valores extremos (outliers) alteran drásticamente la pendiente OLS.

CÓDIGO PYTHON (GOOGLE COLAB / SCIKIT-LEARN)

```python
from sklearn.linear_model import LinearRegression
modelo = LinearRegression()
modelo.fit(X_train, y_train)
predicciones = modelo.predict(X_test)
print('Coeficientes:', modelo.coef_)
print('Intersección:', modelo.intercept_)
```

AJUSTE 2D: RECTA DE REGRESIÓN Y RESIDUOS (OLS)



Algoritmo 1: Regresión Lineal

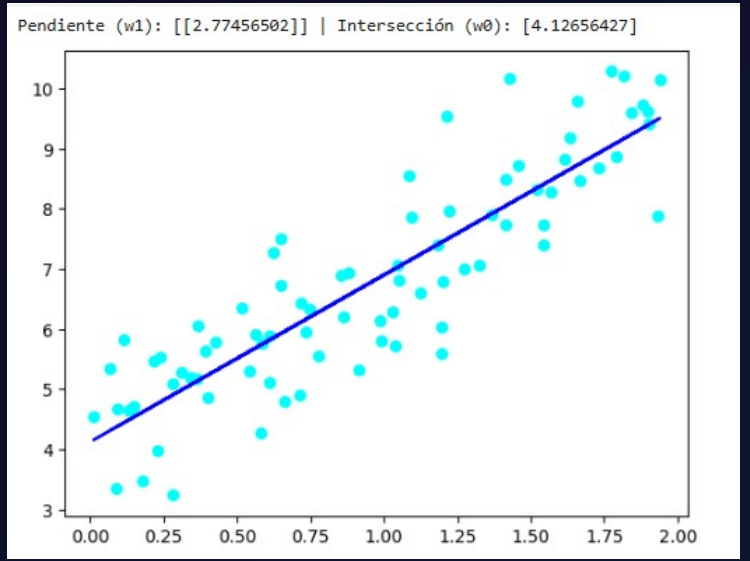
```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# 1.  Creamos datos de prueba dispersos.
# X representa la variable de entrada e y el resultado real con ruido/error.
np.random.seed(42)
X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)

# 2. Separamos datos para entrenamiento (train) y prueba (test).
# Se hace para entrenar al modelo con un grupo y evaluarlo con otro totalmente nuevo.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 3. fit() ejecuta Mínimos Cuadrados Ordinarios (OLS).
# AUTOMATIZACIÓN DE LA FÓRMULA: Toda la matemática de la diapositiva se resuelve aquí.
# El algoritmo calcula de forma automática la combinación lineal y = w0 + w1*x y
# minimiza los errores cuadráticos sin necesidad de hacer los cálculos a mano.
modelo = LinearRegression().fit(X_train, y_train)

# 4. Extraemos los pesos y visualizamos los residuos.
# AUTOMATIZACIÓN DE PARÁMETROS: coef_ e intercept_ extraen automáticamente los pesos w1 y w0.
# La gráfica dibuja la línea OLS final para ver si hay datos atípicos desviando la pendiente.
print('Pendiente (w1):', modelo.coef_, '| Intersección (w0):', modelo.intercept_)
plt.scatter(X_train, y_train, color='cyan')
plt.plot(X_train, modelo.predict(X_train), color='blue')
plt.show()
```



Algoritmo 2: Regresión Logística

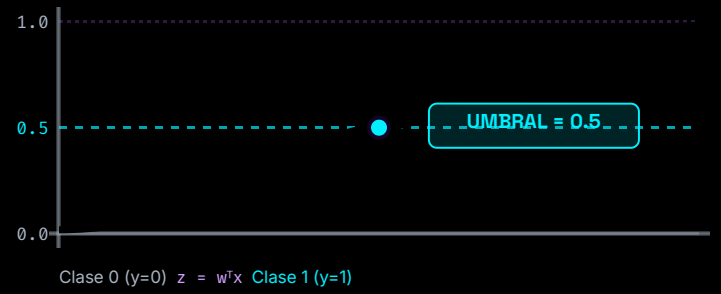
PRINCIPIO MATEMÁTICO

Algoritmo de CLASIFICACIÓN binaria que estima la probabilidad de pertenencia a una clase.

$$P(y=1|x) = 1 / (1 + \exp(-w^T x))$$

Produce probabilidades entre 0 y 1 e interpreta sus coeficientes mediante Odds Ratios (Hosmer et al., 2013).

GRÁFICO: CURVA SIGMOIDE EN S



CÓDIGO PYTHON (GOOGLE COLAB / SCIKIT-LEARN)

```python
# 1. Importar el modelo logístico
from sklearn.linear_model import LogisticRegression

# 2. Instanciar y entrenar el modelo
modelo = LogisticRegression()
modelo.fit(X_train, y_train)

# 3. Predicción de clases y probabilidades
clase_predicha = modelo.predict(X_test)
probabilidades = modelo.predict_proba(X_test)
```

predict_proba() retorna la probabilidad P(y=0) y P(y=1) para cada instancia de prueba.

Algoritmo 2: Regresión Logística

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# ======================================================================
# 1. ORIGEN DE LOS DATOS: Creamos datos de clasificación binaria.
# ======================================================================
# X representa la variable continua e 'y' representa las categorías (0 o 1).
np.random.seed(42)
X = np.random.randn(100, 1) * 3
# Si X es mayor a 0, la clase tiende a ser 1, de lo contrario tiende a ser 0
y = (X > 0).astype(int).ravel()

# MOTIVO DE LA DIVISIÓN: Separamos los datos para entrenar (train) y evaluar (test).
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

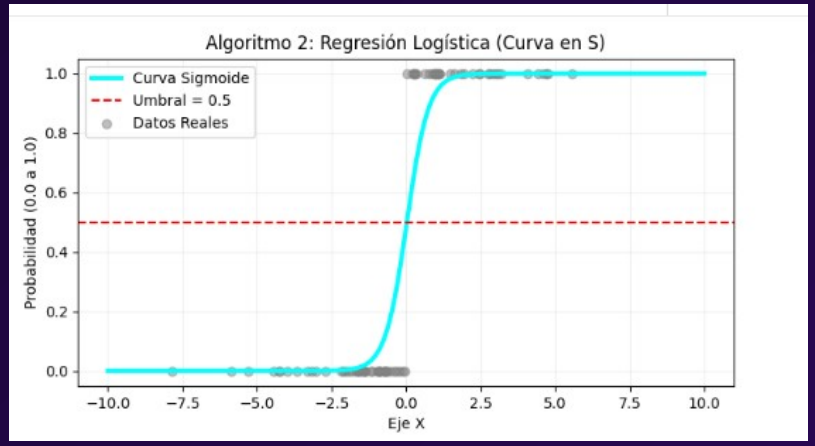

# ======================================================================
# 2. MOTIVO DEL AJUSTE: .fit() ejecuta el entrenamiento logístico.
# ======================================================================
# AUTOMATIZACIÓN DE LA FÓRMULA: Toda la matemática compleja de la probabilidad
# P(y=1|x) = 1 / (1 + exp(-w^T*x)) se resuelve automáticamente en esta línea.
# El algoritmo calcula por sí mismo los coeficientes óptimos sin hacer cálculos manuales.
modelo = LogisticRegression().fit(X_train, y_train)


# ======================================================================
# 3. MOTIVO DE LAS PREDICCIONES: Clasificación y Probabilidades.
# ======================================================================
# AUTOMATIZACIÓN DEL UMBRAL = 0.5: .predict() calcula directamente si es Clase 0 o 1.
# .predict_proba() calcula la probabilidad exacta de pertenecer a cada clase.
clase_predicha = modelo.predict(X_test)
probabilidades = modelo.predict_proba(X_test)


# ======================================================================
# 4. MOTIVO DE LA GRÁFICA: Visualizar la Curva Sigmoide en S.
# ======================================================================
# Se hace para comprobar cómo el modelo convierte números en probabilidades entre 0 y 1.
X_curva = np.linspace(-10, 10, 300).reshape(-1, 1)
# Extraemos solo la probabilidad de la Clase 1 para dibujar la "S"
prob_curva = modelo.predict_proba(X_curva)[:, 1]

plt.figure(figsize=(8, 4))
plt.plot(X_curva, prob_curva, color='cyan', linewidth=3, label='Curva Sigmoide')
plt.axhline(0.5, color='red', linestyle='--', label='Umbral = 0.5') # Línea del umbral
plt.scatter(X_train, y_train, color='gray', alpha=0.5, label='Datos Reales')

plt.title('Algoritmo 2: Regresión Logística (Curva en S)')
plt.xlabel('Eje X')
plt.ylabel('Probabilidad (0.0 a 1.0)')
plt.legend()
plt.grid(True, alpha=0.2)
plt.show()
```



Algoritmo 3: Árboles de Decisión (CART)

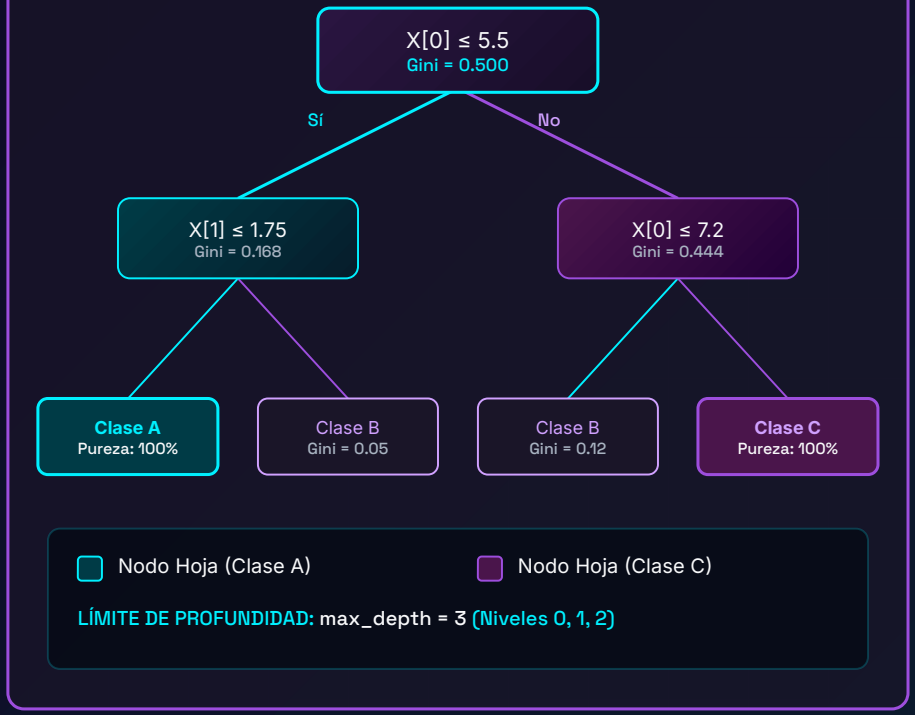
PRINCIPIO MATEMÁTICO

- Representa los datos en reglas jerárquicas si-entonces facilitando la interpretabilidad.
- Criterios de partición basados en Impureza de Gini y Entropía (Breiman et al., 1984).
- Control de modelo: requiere limitar el hiperparámetro max_depth para frenar el sobreajuste.

PYTHON / GOOGLE COLAB (SCIKIT-LEARN)

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
# Configurar y entrenar el modelo CART
modelo = DecisionTreeClassifier(max_depth=3)
modelo.fit(X_train, y_train)
# Visualizar el árbol de decisión
plot_tree(modelo)
```

DIAGRAMA DE ÁRBOL JERÁRQUICO



Algoritmo 3: Árboles de Decisión (CART)

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree

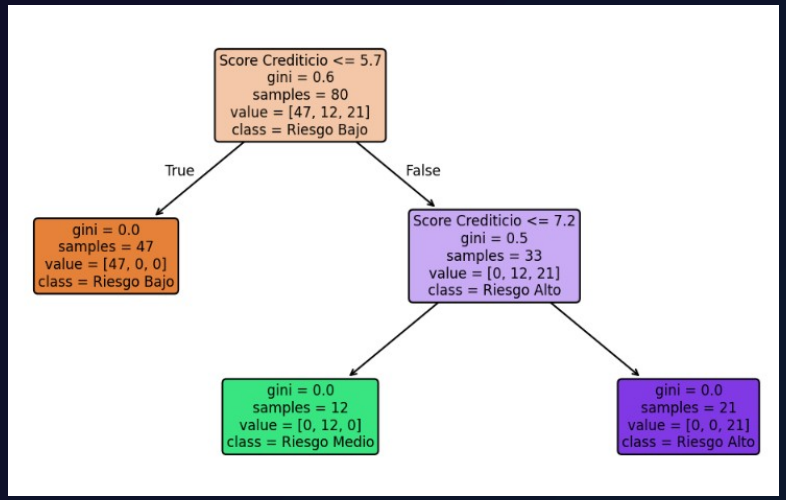
# 1. ORIGEN DE LOS DATOS: Mezclamos clientes de Riesgo Bajo, Medio y Alto
np.random.seed(42)
X = np.random.rand(100, 2) * 10
y = np.zeros(100, dtype=int)
y[(X[:, 0] > 5.5) & (X[:, 0] <= 7.2)] = 1  # Clientes etiquetados como Riesgo Medio
y[X[:, 0] > 7.2] = 2                       # Clientes etiquetados como Riesgo Alto

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)


# ======================================================================
# 2. EL ALGORITMO APLICANDO LA FÓRMULA DE GINI: Gini = 1 - sum(pi^2)
# ======================================================================
# .fit() evalúa la fórmula de Gini en todos los datos para medir el desorden.
# Su único objetivo es separar la mezcla de riesgos hasta que el Gini baje a 0.0.
modelo_arbol = DecisionTreeClassifier(max_depth=3).fit(X_train, y_train)


# ======================================================================
# 3. CÓMO CLASIFICA EL ÁRBOL EN LA GRÁFICA (Paso a Paso)
# ======================================================================
plt.figure(figsize=(10, 5), dpi=150)

# El gráfico plot_tree() dibuja las 3 decisiones automáticas del modelo:
# 1. CAJA RAÍZ: Evalúa si 'Score Crediticio <= 5.7'. Al estar todos mezclados, el Gini es alto (~0.6).
# 2. RAMA IZQUIERDA: Si cumple, los manda aquí. Logra separar la mezcla y el Gini cae a 0.0 (Riesgo Bajo puro).
# 3. RAMA DERECHA: Si no cumple, aplica otro filtro ('Score <= 7.2') para separar Riesgo Medio de Riesgo Alto.
plot_tree(
    modelo_arbol,
    filled=True,
    rounded=True,                  # Cajas redondeadas y limpias para la exposición
    precision=1,                   # Redondea los cortes a un solo decimal (ej. 5.7)
    fontsize=8,
    feature_names=['Score Crediticio', 'Ingresos mensuales'],
    class_names=['Riesgo Bajo', 'Riesgo Medio', 'Riesgo Alto']
)
plt.show()
```



K-Nearest Neighbors (k-NN)

Principio Matemático

Clasificador perezoso (Lazy Learner) basado en instancias que memoriza los datos y clasifica por mayoría de votos entre sus *k* vecinos más cercanos en distancia euclidiana (Cover & Hart, 1967).

⚠ Exige escalado obligatorio con StandardScaler.

DIAGRAMA k-NN: VECINDAD PARA k=1 Y k=3



CÓDIGO PYTHON / GOOGLE COLAB

```python
# 1. Importación de módulos
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

# 2. Escalado obligatorio de características
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# 3. Inicialización y entrenamiento (k=5)
modelo = KNeighborsClassifier(n_neighbors=5)
modelo.fit(X_train_scaled, y_train)
```

K-Nearest Neighbors (k-NN)

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

# 1. ORIGEN DE LOS DATOS: Puntos dispersos en el plano (X) etiquetados en Clase A o B (y)
np.random.seed(42)
X = np.random.rand(100, 2) * 100  # Datos sin escalar (ej. ingresos de 0 a 100k y edad de 0 a 10)
y = (X[:, 0] + X[:, 1] > 100).astype(int) # Clase 0 (Clase A) o Clase 1 (Clase B)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

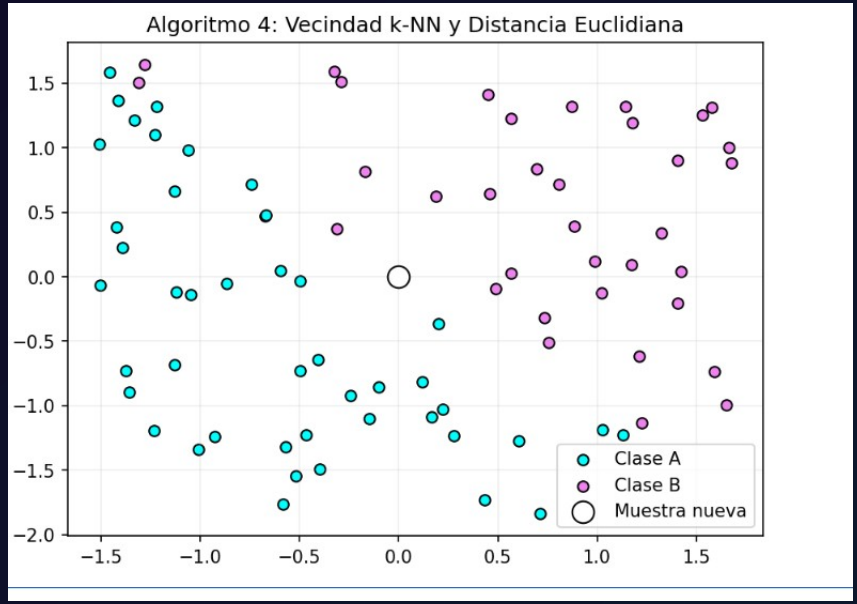
# ======================================================================
# 2. ESCALADO OBLIGATORIO: StandardScaler()
# ======================================================================
# MOTIVO: El algoritmo k-NN mide distancias geométricas (líneas rectas).
# Si una variable tiene números gigantes (ingresos) y otra pequeños (edad),
# la distancia euclidiana se deforma. El escalado equilibra todo al mismo nivel.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ======================================================================
# 3. ENTRENAMIENTO Y VECINDAD: n_neighbors=5 (k=5)
# ======================================================================
# MOTIVO: Es un clasificador perezoso (Lazy Learner) porque .fit() solo memoriza los puntos escalados.
# Al predecir, medirá la distancia euclidiana hacia los 5 vecinos más cercanos y votarán por mayoría.
modelo = KNeighborsClassifier(n_neighbors=5)
modelo.fit(X_train_scaled, y_train)

# ======================================================================
# 4. GRÁFICA DEL DIAGRAMA k-NN: Vecindad en acción
# ======================================================================
# Dibujamos los puntos memorizados para entender cómo votan en su cercanía
plt.figure(figsize=(7, 5), dpi=150)
plt.scatter(X_train_scaled[y_train==0, 0], X_train_scaled[y_train==0, 1], color='cyan', edgecolors='black', label='Clase A')
plt.scatter(X_train_scaled[y_train==1, 0], X_train_scaled[y_train==1, 1], color='violet', edgecolors='black', label='Clase B')

# Dibujamos un punto blanco aleatorio para simular la "Muestra a clasificar" de tu diagrama
plt.scatter(0, 0, color='white', s=150, edgecolors='black', zorder=5, label='Muestra nueva')

plt.title('Algoritmo 4: Vecindad k-NN y Distancia Euclidiana')
plt.legend()
plt.grid(True, alpha=0.2)
plt.show()
```



Aplicaciones y Limitaciones

Aplicaciones

- Adopción masiva en medicina para diagnósticos.
- Finanzas paa evaluación de riesgo crediticio.
- Ciberseguridad para detección de intrusiones.
- Procesamiento de texto para filtrado de spam.

Sarker (2021)

Limitaciones

- Requiere enormes volúmenes de datos etiquetados.
- Susceptible al principio 'Garbage In, Garbage Out'.
- Sufre de sobreajuste (Overfitting) al memorizar ruido.
- Exige regularización Lasso/Ridge para patrones.

Hastie et al. (2009)



Mapa Mental del Aprendizaje Supervisado

NODO CENTRAL
APRENDIZAJE SUPERVISADO

01. CONDICIONES

- **Datos Etiquetados:** Pares (X, Y) con ground truth.
- **Volumen:** Muestras suficientes para generalizar.
- **Objetivo:** Función de mapeo $f(X) \rightarrow Y$.

02. TIPOS DE PROBLEMA

- **Clasificación:** Predicción de etiquetas discretas (Binaria / Multiclase).
- **Regresión:** Predicción de valores numéricos continuos.

03. CAJA BLANCA (EXPLICABLES)

- Regresión Lineal / Logística: Mapeo directo.
- Árboles de Decisión: Reglas Lógicas "If-Then".
- k-NN: Clasificación por proximidad espacial.

04. CAJA NEGRA (COMPLEJOS)

- **Redes Neuronales:** Deep Learning multi-capa.
- **Random Forest:** Ensamble Bagging de árboles.
- **XGBoost:** Gradient Boosting secuencial.

05. CASOS DE USO

- **Diagnóstico Médico:** Detección de patologías.
- **Predicción de Precios:** Estimación de inmuebles y finanzas.

06. MÉTRICAS CLAVE

- **Clasificación:** Exactitud, Precisión, Recall, F1.
- **Regresión:** MSE, RMSE, MAE, R^2.

⚠ **Data Leakage:** Riesgo crítico a prevenir en validación.

ARTÍCULO 1: DESERCIÓN UNIVERSITARIA

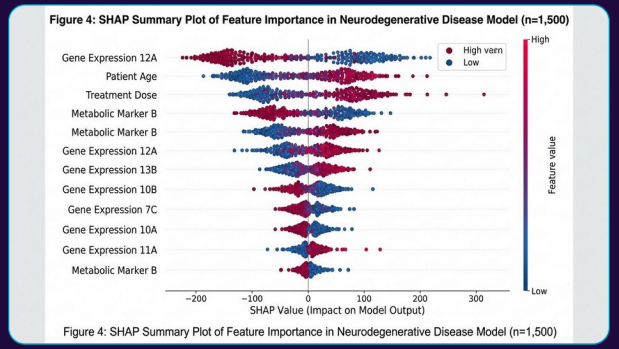
FACTORS, PREDICTION, EXPLAINABILITY, AND SIMULATING UNIVERSITY DROPOUT

Problema y Enfoque

Marco metodológico para predecir abandono estudiantil temprano y transparentar factores socioeconómicos/académicos para tutorías.

Metodología y Resultados

- Algoritmos: Regresión Logística, Random Forest, XGBoost y LightGBM con valores SHAP.
- Performance: XGBoost alcanzó ROC-AUC > 0.91.
- SHAP: El rendimiento del primer semestre pesa 3 veces más que lo demográfico.



Visualización de Explicabilidad SHAP

Cita: Quimiz-Moreira et al. (2025) | Link: https://doi.org/10.3390/computation13080198

FRONTERA CIENTÍFICA SCOPUS UNMSM

Deserción Universitaria y Explicabilidad SHAP

SCOPUS

REFERENCIA CIENTÍFICA

Quimiz-Moreira, Delgadillo, Maculan & Mauricio (2025)

Computation, 13(8), 198. MDPI / Scopus

MODELADO Y METODOLOGÍA

Predicción de la deserción en educación superior pública utilizando ensambles XGBoost y LightGBM integrados con valores SHAP (teoría de juegos) para abrir la caja negra.

MÉTRICA ROC-AUC	RECALL DETECCIÓN
> 0.91	88%

HALLAZGO CLAVE SHAP

El análisis de explicabilidad demostró que las **notas del primer ciclo** influyen:

3x más que el factor socioeconómico

en el comportamiento de deserción en educación pública.

